Introducció a la màquina senzilla

M0371 - Bloc 1 - Fonaments de maquinari

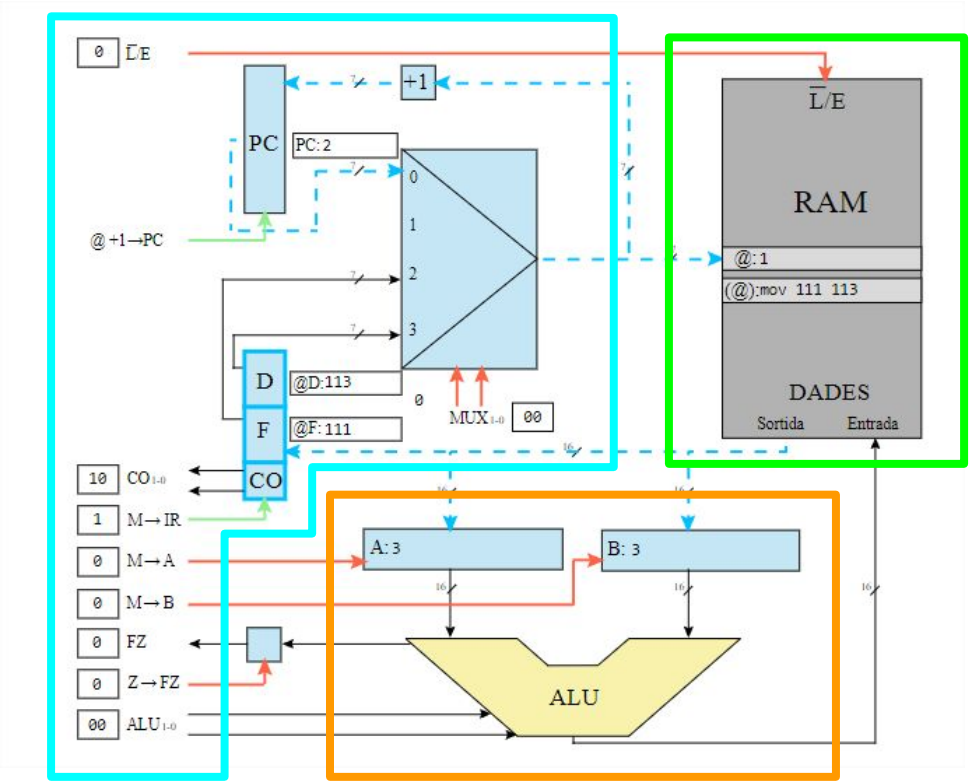
CC BY SA



La màquina senzilla

- Estructura de processador molt bàsica i senzilla.
- Permet entendre resumidament com funciona una CPU.
- Eina desenvolupada per Bryan Pierre en el seu TFG a la Universitat de Girona.
- Ús públic (GNU General Public License v3.0)
- Enllaç.

Estructura de la màquina senzilla

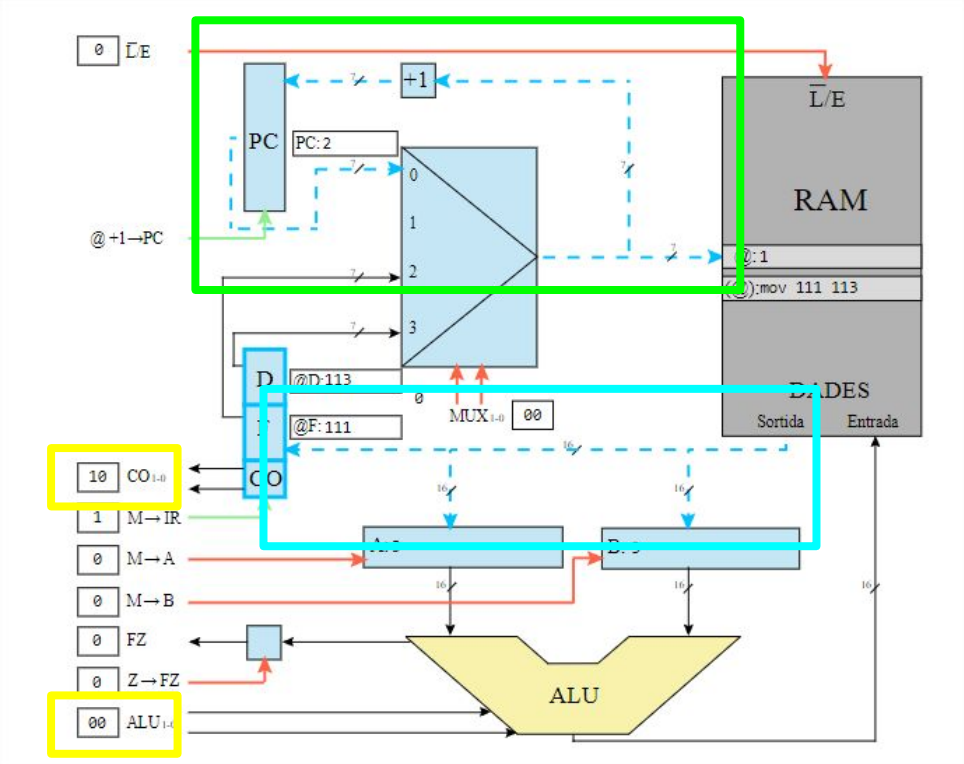


Memòria

Unitat aritmètico lògica (ALU)

Unitat de control i registres de control

Estructura de la màquina senzilla



Bus d'adreces **(7 bits)**, quantes posicions de memòria?

Unitat de control i registres de control **(16 bits)**

Codis d'operació **(2 bits).** Quantes operacions podem codificar?

Registres i bits de control

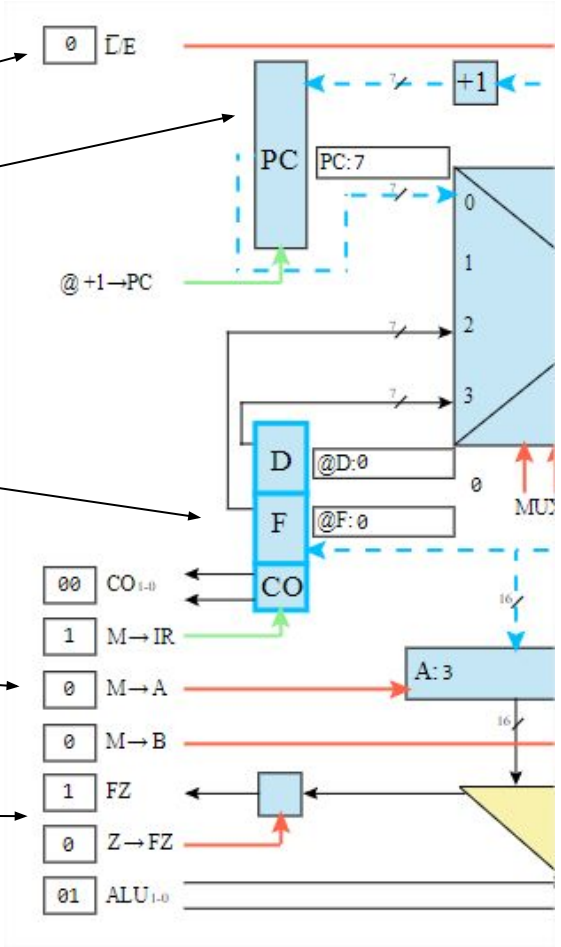
Bit indicador d'**escriptura** (1) o **lectura** (0)

Comptador del programa (següent instrucció?)

Adreça **destí** (D) - 7 bits
Adreça **origen** (F) - 7 bits
Codi d'**operació** (CO) - 2 bits

Bits de **control** (flags) per marcar si cal **gravar** els valors que ens arriben dels busos de dades provinents de la memòria:
M → A, M → B, M → IR

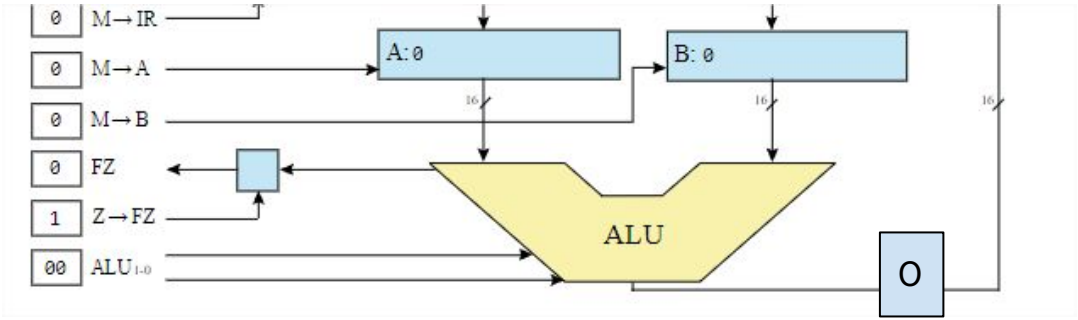
Bit de control **FZ**, calculat en base d'operands i operació de l'ALU. Només es registra si s'activa el bit **Z**.



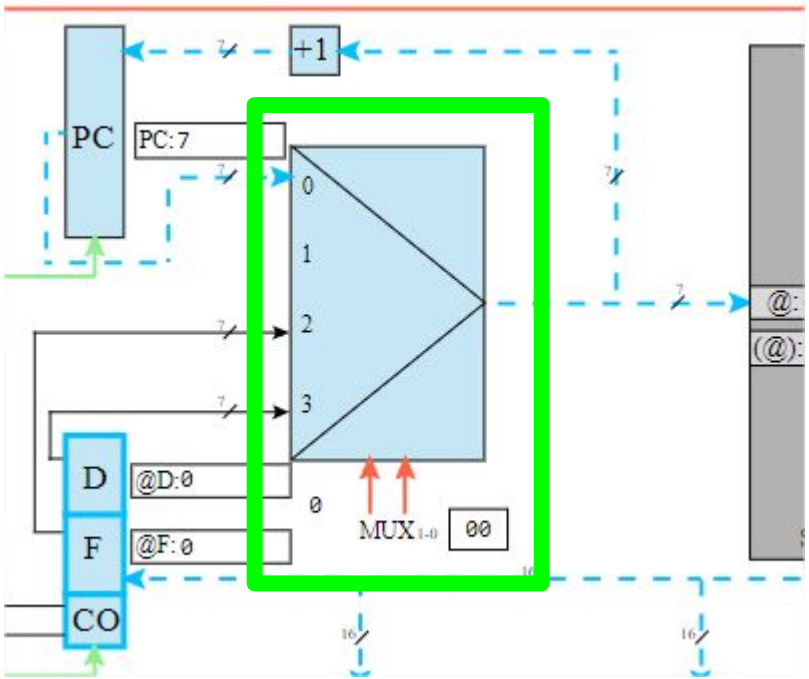
Tots aquests bits de control els gestiona la **UNITAT de CONTROL.**

Instruccions acceptades

Instrucció	Càlcul	Sortida (ALU)	Flag Z (ALU)
ADD F, D	D = F + D	O = A (D) + B (F),	FZ = (A+B==0) ?
MOV F, D	D = F	O = B (F),	FZ = (B == 0) ?
CMP F, D	F == D ?	-	FZ = (A == B) ?
BEQ D (Branch if Equal → “salta si són iguals”)	FZ == 1 ? → Saltem a D	-	



El multiplexor!

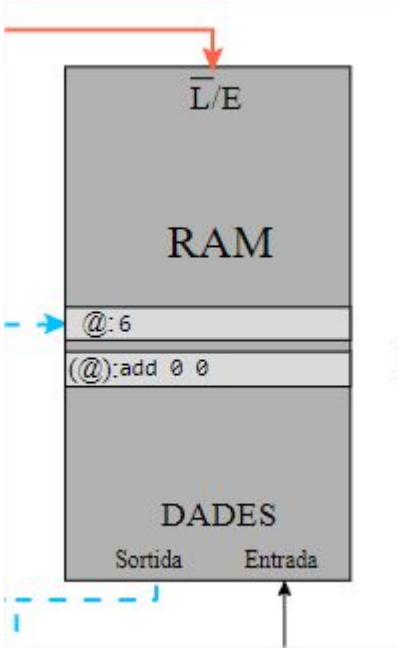


El valor en **binari** de **MUX**, indica al multiplexor quina de les entrades **ha de deixar passar**.

Ens permetrà **alternar** que volem fer servir en els diferents **cicles** de rellotge de cada instrucció.
Circuit d'un MUX de 4 entrades.

MUX	Sortida
00b = 0	Següent instrucció (PC)
01b = 1	0 (no es fa servir)
10b = 2	Adreça d'origen (F)
11b = 3	Adreça de destí (D)

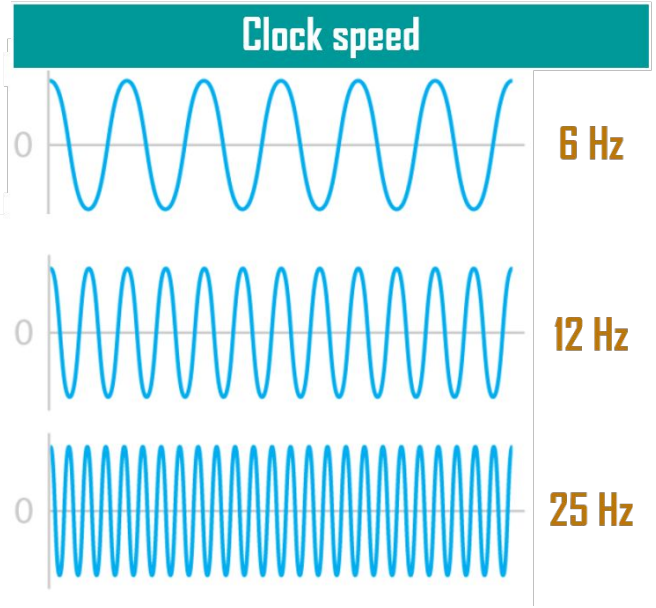
La memòria



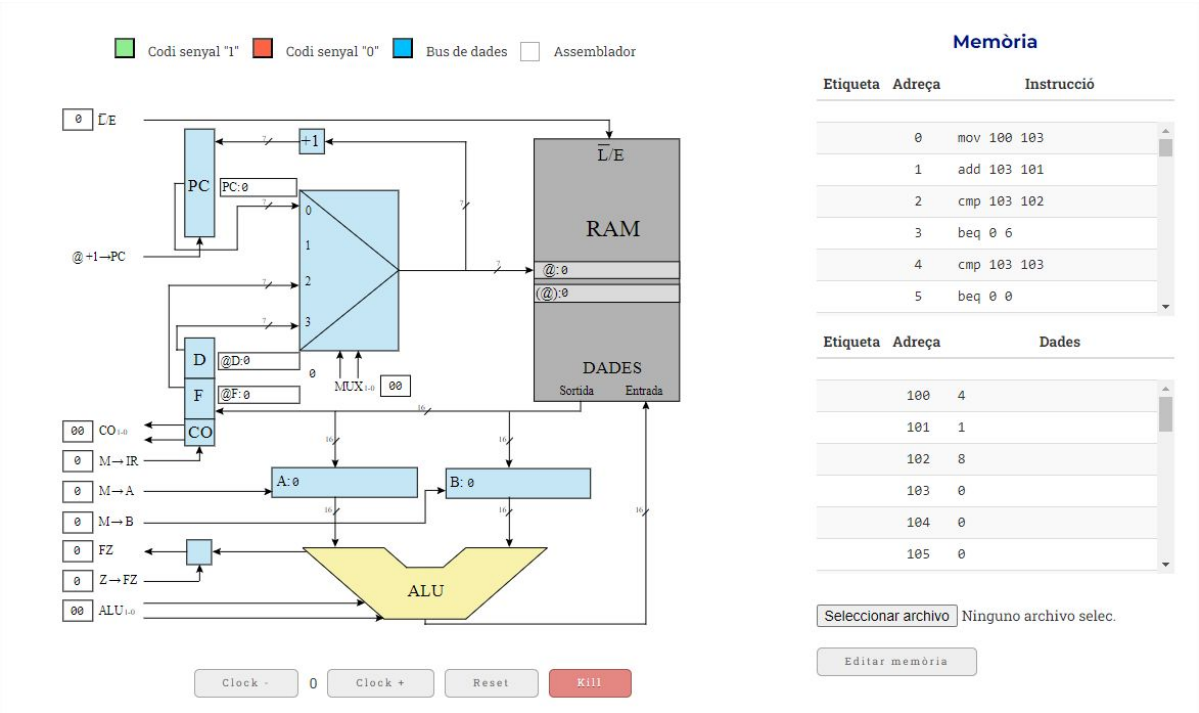
- Cal indicar-li si volem **llegir (0)** o **escriure (1).**
- Li hem d'indicar a quina **posició** de memòria volem accedir, amb el **bus d'adreces**.
  - Si **llegim,** es bolcarà cap a la sortida a través del **bus de dades**.
  - Si **escrivim,** guardarà el valor de l'entrada a la posició indicada.

Execució d'instruccions (resum simplificat)

1. **Cercar** següent instrucció (**Fetch**)
2. Analitzar què cal fer amb la instrucció (**Decode**)
3. Carregar **operands**
4. **Executar** operació a l'ALU
5. Gestionar **resultats**
6. **Següent** instrucció



Fem una simulació!



Instruccions:

```
mov 100,103;
add 101,103;
cmp 103,102;
beq 6;
cmp 103,103;
beq 0;
```

Dades:

```
100: 4
101: 1
102: 8
```

Exemple

Instruccions:

```
mov 100,103;
add 101,103;
cmp 103,102;
beq 6;
cmp 103,103;
beq 0;
```

Dades:

```
100: 4
101: 1
102: 8
```

Iteració	Instruccions executades	Valor de [103]	Comparació amb [102]=8	Acció següent
0	`mov 100,103` → carrega 4 `add 101,103` → 4+1=5	5	5 ≠ 8	Continua amb cmp 103,103 → beq 0 (torna a començar)
1	`mov 100,103` → carrega 4 `add 101,103` → 4+1=5	5	5 ≠ 8	Continua amb cmp 103,103 → beq 0 (torna a començar)
2	`mov 100,103` → carrega 4 `add 101,103` → 4+1=5	5	5 ≠ 8	Continua amb cmp 103,103 → beq 0 (torna a començar)
3	`mov 100,103` → carrega 4 `add 101,103` → 4+1=5	5	5 ≠ 8	Continua amb cmp 103,103 → beq 0 (torna a començar)

Exemple

El programa anterior és un bucle infinit! Com fem que finalitzi?

Instruccions:

```
mov 100,103;
add 101,103;
cmp 103,102;
beq 6;
cmp 103,103;
beq 1;
```

Dades:

```
100: 4
101: 1
102: 8
```

Iteració	Instruccions executades	Valor de [103]	Comparació amb [102]=8	Acció següent
0	`mov 100,103` → carrega 4 `add 101,103` → 4+1=5	5	5 ≠ 8	Continua amb cmp 103,103 → beq 1 (torna a començar)
1	`mov 100,103` → carrega 5 `add 101,103` → 5+1=6	6	6 ≠ 8	Continua amb cmp 103,103 → beq 1 (torna a començar)
2	`mov 100,103` → carrega 6 `add 101,103` → 6+1=7	7	7 ≠ 8	Continua amb cmp 103,103 → beq 1 (torna a començar)
3	`mov 100,103` → carrega 7 `add 101,103` → 7+1=8	8	8 = 8	cmp 103, 102 (8=8) i salta (beq 6) a l'adreça 6, acabant el programa